



UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MÉXICO
FACULTAD DE INGENIERÍA

SISTEMAS DIGITALES
DETECCIÓN DE INTRUSOS
EQUIPO 4

INTEGRANTES
Molina Aguilar Enrique
Jimenez Perez Axel

23 DE ABRIL DEL 2025

4. Sala 4: Sistema de detección de intrusos: Un sistema de seguridad que reacciona de manera asíncrona a eventos como aperturas de puertas o movimientos sospechosos.

1. Definición de Estados

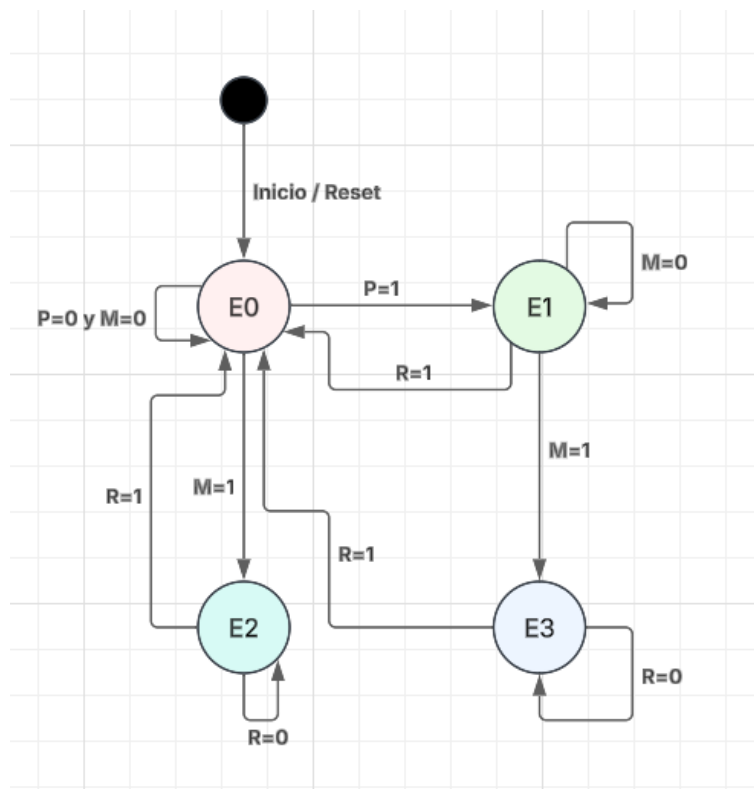
Estados:

- **E0:** Sistema en reposo (esperando eventos)
- **E1:** Detección de puerta abierta
- **E2:** Detección de movimiento
- **E3:** Alarma activa

Entradas:

- **P:** Sensor de puerta (1 = abierta, 0 = cerrada)
- **M:** Sensor de movimiento (1 = movimiento detectado, 0 = sin movimiento)
- **R:** Reset (1 = reinicia sistema a E0)

2. Diagrama de Estados



3. Tabla de Transiciones

Estado Actual	P	M	R	Estado Siguiente	Salida
E0	0	0	0	E0	Sistema en reposo: puerta cerrada y sin movimiento.
E0	1	0	0	E1	Alerta: puerta abierta, pero sin movimiento.
E0	0	1	0	E2	Alerta: movimiento detectado con puerta cerrada.
E0	*	*	1	E0	Reinicio: sistema vuelve a reposo (puerta cerrada, sin movimiento).
E1	*	1	0	E3	ALARMA ACTIVADA: puerta abierta y movimiento sospechoso detectado.
E1	*	*	1	E0	Reinicio: sistema vuelve a reposo (se canceló alerta de puerta abierta).
E2	*	*	1	E0	Reinicio: sistema vuelve a reposo (se canceló alerta de movimiento).
E2	*	*	0	E2	Alerta continua: movimiento detectado (puerta abierta/cerrada).
E3	*	*	0	E3	ALARMA CONTINÚA: activada por movimiento sospechoso (puerta abierta/cerrada).
E3	*	*	1	E0	Reinicio: alarma desactivada, sistema vuelve a reposo.

Nota: (*) significa cualquier valor (0 o 1)

4. Implementación Lógica

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity IntrusionSystem is
    Port ( P : in  STD_LOGIC;
          M : in  STD_LOGIC;
          R : in  STD_LOGIC;
          CLK : in  STD_LOGIC;
          state_out : out STD_LOGIC_VECTOR(1 downto 0));
end IntrusionSystem;

architecture Behavioral of IntrusionSystem is
    type State_Type is (E0, E1, E2, E3);
    signal state, next_state : State_Type;
begin

    process(CLK, R)
    begin
        if R = '1' then
            state <= E0;
        elsif rising_edge(CLK) then
            state <= next_state;
        end if;
    end process;

    process(state, P, M)
    begin
        case state is
            when E0 =>
                if P = '1' then
                    next_state <= E1;
                elsif M = '1' then
                    next_state <= E2;
                else
                    next_state <= E0;
                end if;
            when E1 =>
                if M = '1' then
                    next_state <= E3;
                else
                    next_state <= E1;
                end if;
            when E2 =>
                next_state <= E2;
            when E3 =>
                next_state <= E3;
        end case;
    end process;

    -- Estado de salida codificado
    state_out <= "00" when state = E0 else
        "01" when state = E1 else
        "10" when state = E2 else
        "11";

end Behavioral;
```